

bruno

April 28, 2026

1 DL2 - Sheet 02: Attention

```
[1]: !pip install transformers
```

Requirement already satisfied: transformers in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (5.5.0)

Requirement already satisfied: huggingface-hub<2.0,>=1.5.0 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from transformers) (1.9.0)

Requirement already satisfied: numpy>=1.17 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from transformers) (2.4.2)

Requirement already satisfied: packaging>=20.0 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from transformers) (25.0)

Requirement already satisfied: pyyaml>=5.1 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from transformers) (6.0.3)

Requirement already satisfied: regex>=2025.10.22 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from transformers) (2026.2.19)

Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from transformers) (0.22.2)

Requirement already satisfied: typer in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from transformers) (0.24.0)

Requirement already satisfied: safetensors>=0.4.3 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from transformers) (0.7.0)

Requirement already satisfied: tqdm>=4.27 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from transformers) (4.67.1)

Requirement already satisfied: filelock>=3.10.0 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from huggingface-hub<2.0,>=1.5.0->transformers) (3.20.3)

Requirement already satisfied: fsspec>=2023.5.0 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from

huggingface-hub<2.0,>=1.5.0->transformers) (2026.1.0)
 Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from huggingface-hub<2.0,>=1.5.0->transformers) (1.4.3)
 Requirement already satisfied: httpx<1,>=0.23.0 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from huggingface-hub<2.0,>=1.5.0->transformers) (0.28.1)
 Requirement already satisfied: typing-extensions>=4.1.0 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from huggingface-hub<2.0,>=1.5.0->transformers) (4.15.0)
 Requirement already satisfied: anyio in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers) (4.12.1)
 Requirement already satisfied: certifi in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers) (2026.1.4)
 Requirement already satisfied: httpcore==1.* in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers) (1.0.9)
 Requirement already satisfied: idna in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers) (3.11)
 Requirement already satisfied: h11>=0.16 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers) (0.16.0)
 Requirement already satisfied: click>=8.2.1 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from typer->transformers) (8.3.1)
 Requirement already satisfied: shellingham>=1.3.0 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from typer->transformers) (1.5.4)
 Requirement already satisfied: rich>=12.3.0 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from typer->transformers) (14.3.3)
 Requirement already satisfied: annotated-doc>=0.0.2 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from typer->transformers) (0.0.4)
 Requirement already satisfied: markdown-it-py>=2.2.0 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from rich>=12.3.0->typer->transformers) (4.0.0)
 Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from rich>=12.3.0->typer->transformers) (2.19.2)
 Requirement already satisfied: mdurl~=0.1 in /nix/store/as2mhsq2wdq6kbb3xbhbiiya88izn00a-python3-3.13.12-env/lib/python3.13/site-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->typer->transformers) (0.1.2)

2 Intro: Transformers for Sequence Classification

In this weeks notebook we will implement our own Transformer model from scratch. Typical Transformers (we focus here on the encoding of sentences for the purpose of sentiment classification, the decoder used in sequence2sequences Transformers has a very similar structure) can be broken down into the following components:

1. Embedding: An embedding layer that transforms word tokens into vector representations.
2. Encoder: The encoder block consists of several multi-headed attention blocks
 - 2.1. Attention block 1
 - 2.2. Attention block 2
 - ...
 - 2.L. Attention block L
3. Pooling: The final output of the encoder computes one vector representation for each token. In order to summarize this information for classification.
4. Classification: A standard MLP-layer is used for classification and outputs probabilities for the most likely predicted class.

2.1 Preparation: From sentences to tokens

First we will use huggingface's tokenizer to go from words to indices in the vocabulary. In the following we will focus on the distilBERT model:

```
[2]: from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
sentence = "This was one of the best movies I have ever seen."
inputs = tokenizer(sentence, return_tensors = 'pt')
print(inputs)
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

config.json: 0%| | 0.00/483 [00:00<?, ?B/s]

tokenizer_config.json: 0%| | 0.00/48.0 [00:00<?, ?B/s]

vocab.txt: 0.00B [00:00, ?B/s]

tokenizer.json: 0.00B [00:00, ?B/s]

```
{'input_ids': tensor([[ 101, 2023, 2001, 2028, 1997, 1996, 2190, 5691, 1045,
2031, 2412, 2464,
1012, 102]]), 'token_type_ids': tensor([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0]]), 'attention_mask': tensor([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1]])}
```

```
[44]: # Download pretrained weights
# ! wget https://tubcloud.tu-berlin.de/s/GfEoq4r8Sgb2727/download/distilbert.pt
```

```
[45]: from torch import nn
import torch
import math
import torch

class Config(object):
    def __init__(self):
        self.n_heads = 12
        self.n_layers = 6
        self.pad_token_id = 0
        self.dim = 768
        self.hidden_dim = 3072
        self.max_position_embeddings = 512
        self.vocab_size = 30522
        self.eps = 1e-12

        self.attention_head_size = int(self.dim / self.n_heads)
        self.all_head_size = self.n_heads * self.attention_head_size

        self.n_classes = 2
        self.device = 'cpu'

config = Config()
```

3 1. Embedding Layer (5p)

First, you will have to implement the Embedding layer.

1. forward: compute the output embeddings from the input_embeds and position_embeds. (Think about how they are merged.)

```
[46]: torch.manual_seed(0) # set seed for reproducible random initialization of
    ↪ weights

class Embeddings(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.word_embeddings = nn.Embedding(config.vocab_size, config.dim,
    ↪ padding_idx=config.pad_token_id)
        self.position_embeddings = nn.Embedding(config.max_position_embeddings,
    ↪ config.dim)
        self.LayerNorm = nn.LayerNorm(config.dim, eps=config.eps)

    def forward(self, input_ids: torch.Tensor) -> torch.Tensor:
        """
        Parameters:
            input_ids (torch.Tensor):
```

```

        torch.tensor(bs, max_seq_length) The token ids to embed.
        Returns: torch.tensor(bs, max_seq_length, dim) The embedded tokens
        (plus position embeddings)
        """

        # Embedding the input ids
        input_embeds = self.word_embeddings(input_ids) # (bs, max_seq_length,
        dim)
        seq_length = input_embeds.size(1)

        # Creating and embedding the position ids
        position_ids = torch.arange(seq_length, dtype=torch.long,
        device=input_ids.device) # (max_seq_length)
        position_ids = position_ids.unsqueeze(0).expand_as(input_ids) # (bs,
        max_seq_length)
        position_embeddings = self.position_embeddings(position_ids) # (bs,
        max_seq_length, dim)

        # Compute the output embeddings
        # 1. START YOUR CODE HERE #

        embeddings = input_embeds + position_embeddings

        # 1. START YOUR CODE HERE #

        embeddings = self.LayerNorm(embeddings) # (bs, max_seq_length, dim)
        return embeddings

embedding_layer = Embeddings(config)
# Test if your embedding layer computes an output
embeddings = embedding_layer(inputs['input_ids'])

embeddings

```

```

[46]: tensor([[[[ 0.7850, -0.1792, -0.7408, ..., -0.3442, -0.2749, -0.4575],
  [ 0.3348, -0.6993,  0.9561, ...,  0.2761, -1.1587,  0.7851],
  [ 0.2848, -0.3849,  1.1479, ...,  0.0363, -0.7349,  1.1516],
  ...,
  [ 0.0217, -0.4367,  0.5442, ...,  0.3884, -0.7516,  0.6763],
  [ 0.6787,  0.4753,  0.0463, ...,  1.3417, -1.0327,  1.0803],
  [-0.2996,  0.0238,  0.8504, ..., -1.1545, -0.5847, -0.9615]]],
grad_fn=<NativeLayerNormBackward0>)]

```

4 2. Attention Block (20 points)

The next step is writing the attention block. It mainly consists of the self-attention function (that you have analyzed in the first part of the exercise sheet), a layer normalization followed by an

additional projection layer.

Please add the missing code:

1. `__init__`: Add the Linear projections for the query, key and value functions.
2. `forward`: Write the main self-attention function

```
[51]: torch.manual_seed(0)

class AttentionBlock(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.config = config

        # self-attention components

        # 1. START YOUR CODE HERE #

        R = config.n_heads
        d = config.dim
        self.q_lin = nn.Linear(d,d)
        self.k_lin = nn.Linear(d,d)
        self.v_lin = nn.Linear(d,d)
        self.softmax = nn.Softmax(dim=-1)

        # 1. END YOUR CODE HERE

        self.out_lin = nn.Linear(in_features=config.dim, out_features=config.
        ↪dim, bias=True)
        self.sa_layer_norm = nn.LayerNorm(normalized_shape=config.dim,
        ↪eps=config.eps)

        # feed-forward network
        self.lin1 = nn.Linear(in_features=config.dim, out_features=3072,
        ↪bias=True)
        self.lin2 = nn.Linear(in_features=3072, out_features=config.dim,
        ↪bias=True)
        self.output_layer_norm = nn.LayerNorm(normalized_shape=config.dim,
        ↪eps=config.eps)

    def forward(self, hidden_states):

        def shape(x):
            """ separate heads """
            return x.view(1, -1, 12, 64).transpose(1, 2)

        def unshape(x):
```

```

        """ group heads """
        return x.transpose(1, 2).contiguous().view(1, -1, 12 * 64)

bs=hidden_states.shape[0]
n_nodes= hidden_states.shape[1]

query=key=value=hidden_states
q = self.q_lin(query)
k = self.k_lin(key)
v = self.v_lin(value)

# Separating the heads
q = shape(q)  # (bs, n_heads, q_length, dim_per_head)
k = shape(k)  # (bs, n_heads, k_length, dim_per_head)
v = shape(v)  # (bs, n_heads, k_length, dim_per_head)

# Normalizing the query-tensor
q = q / math.sqrt(q.shape[-1])

# 2. START YOUR CODE HERE #

# Compute attention scores
w = q @ k.transpose(-2, -1)

# Transform the scores into probability distribution via softmax
W = self.softmax(w)

# Compute the weighted representation of the value-tensor (aka context)
y = W @ v
context = y

# 2. END YOUR CODE HERE #

# Merging the heads again
context = unshape(context)  # (bs, q_length, dim)

# Additional projection of the context to get the output of the
↪ self-attention block
sa_output = self.out_lin(context)
sa_output = self.sa_layer_norm(sa_output + hidden_states)

# Feed-forward network to compute the attention block output
x = self.lin1(sa_output)
x = nn.functional.gelu(x)
ffn_output = self.lin2(x)
ffn_output = self.output_layer_norm(ffn_output + sa_output)

```

```

        return ffn_output, W

block = AttentionBlock(config)
# Test if your attention block computes an output
block_output = block(embeddings)

block_output

```

```

[51]: (tensor([[[ 0.7114, -0.0722, -0.5230, ..., -0.1903, -0.7819, -0.5930],
 [ 0.2216, -0.9576,  0.9970, ..., -0.0926, -1.1536,  0.8901],
 [ 0.4497, -0.6879,  1.1513, ...,  0.0209, -0.9052,  1.1951],
 ...,
 [-0.1305, -0.3480,  0.5797, ...,  0.3764, -0.8779,  0.5787],
 [ 0.6187,  0.2542,  0.1652, ...,  1.0826, -1.0824,  0.9224],
 [-0.1873,  0.0490,  0.7358, ..., -1.2114, -0.6134, -0.7558]]],
      grad_fn=<NativeLayerNormBackward0>),
      tensor([[[[0.0434, 0.0909, 0.0583, ..., 0.1567, 0.0598, 0.0668],
 [0.0969, 0.0588, 0.0794, ..., 0.0579, 0.0654, 0.0741],
 [0.0529, 0.0764, 0.0608, ..., 0.0522, 0.0904, 0.0868],
 ...,
 [0.0657, 0.0971, 0.1063, ..., 0.0665, 0.0505, 0.0848],
 [0.0883, 0.0417, 0.1146, ..., 0.1033, 0.0529, 0.0934],
 [0.1194, 0.0599, 0.0607, ..., 0.0505, 0.0614, 0.0591]],

 [[0.0197, 0.0776, 0.0725, ..., 0.0774, 0.0452, 0.1007],
 [0.0580, 0.0531, 0.1805, ..., 0.0595, 0.0481, 0.0488],
 [0.0740, 0.0582, 0.0836, ..., 0.0834, 0.0513, 0.0613],
 ...,
 [0.0609, 0.0808, 0.0940, ..., 0.0750, 0.0579, 0.0563],
 [0.0940, 0.0579, 0.0625, ..., 0.0605, 0.0566, 0.0800],
 [0.0347, 0.1090, 0.0651, ..., 0.0415, 0.0822, 0.0602]],

 [[0.0713, 0.0696, 0.0610, ..., 0.0518, 0.0681, 0.0424],
 [0.0995, 0.0449, 0.0531, ..., 0.0194, 0.0455, 0.0827],
 [0.0792, 0.0897, 0.0594, ..., 0.0341, 0.0447, 0.1238],
 ...,
 [0.0578, 0.1115, 0.0490, ..., 0.0912, 0.0793, 0.0916],
 [0.1088, 0.0735, 0.0675, ..., 0.0452, 0.0388, 0.0921],
 [0.0817, 0.0562, 0.0701, ..., 0.0554, 0.0796, 0.0604]],

 ...,

 [[0.0477, 0.0677, 0.0459, ..., 0.0571, 0.0909, 0.0790],
 [0.0645, 0.0570, 0.0538, ..., 0.1012, 0.0618, 0.0555],
 [0.0779, 0.0998, 0.0554, ..., 0.0543, 0.0790, 0.0684],
 ...,
 [0.0735, 0.0531, 0.0597, ..., 0.0728, 0.0636, 0.0572],

```

```

[0.0682, 0.0776, 0.0640, ..., 0.0703, 0.0635, 0.0671],
[0.0599, 0.0946, 0.0948, ..., 0.1572, 0.0878, 0.0454]],

[[0.0749, 0.0834, 0.0690, ..., 0.0588, 0.0469, 0.0702],
[0.0808, 0.0887, 0.0401, ..., 0.1106, 0.0904, 0.0474],
[0.0630, 0.0525, 0.0656, ..., 0.0826, 0.0535, 0.0649],
...,
[0.0954, 0.0685, 0.0830, ..., 0.0772, 0.0390, 0.0527],
[0.0751, 0.1311, 0.0592, ..., 0.0547, 0.0957, 0.0692],
[0.0841, 0.0517, 0.0462, ..., 0.0726, 0.0990, 0.0743]],

[[0.0448, 0.0504, 0.0980, ..., 0.0628, 0.0405, 0.0326],
[0.0600, 0.0585, 0.0775, ..., 0.0950, 0.0999, 0.0700],
[0.0575, 0.1256, 0.0897, ..., 0.0512, 0.0349, 0.1047],
...,
[0.0649, 0.0609, 0.0928, ..., 0.0484, 0.0931, 0.0536],
[0.0668, 0.0944, 0.0297, ..., 0.0597, 0.0402, 0.0861],
[0.0919, 0.0231, 0.0281, ..., 0.0641, 0.0864, 0.0764]]]],
grad_fn=<SoftmaxBackward0>))

```

5 3. Building the model (15 points)

Now, we can finally put it all together. For this, please, write the following missing code:

1. `__init__`: Add the attention layers (the encoder) to the model
2. `forward`: Add the missing code for sequentially looping through the attention layers
3. `forward`: Add the classifier, which consists of [1. `pre_classifier`, 2. `ReLU` activation, 3. `classifier`] and returns the logit scores

```

[66]: torch.manual_seed(0)

class DistillBertAttention(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.config = config
        self.n_layers=config.n_layers

        # embedding
        self.embeddings = Embeddings(config)

        # encoder
        # 1. START YOUR CODE HERE #

        self.attention_layers = nn.ModuleList([AttentionBlock(config) for _ in
        ↪range(config.n_layers)])

        # 1. END YOUR CODE HERE #

```

```

        # classification
        self.pre_classifier = nn.Linear(in_features=config.dim,
        ↪out_features=config.dim, bias=True)
        self.classifier = nn.Linear(in_features=config.dim,
        ↪out_features=config.n_classes, bias=True)

        self.attention_probs = {i: [] for i in range(config.n_layers)}

    def forward(self, input_ids):
        """
        Parameters:
            input_ids (torch.Tensor): torch.tensor(bs, max_seq_length) The
        ↪token ids to embed.
        Returns: torch.tensor(bs, n_classes) The computed logit scores for each
        ↪class.
        """

        # Computing the embeddings
        hidden_states = self.embeddings(input_ids=input_ids).to(self.config.
        ↪device)

        # Iteratively going through the attention layers
        encoder_input = hidden_states
        # 2. START YOUR CODE HERE #

        output = hidden_states
        for i, layer in enumerate(self.attention_layers):
            output, weights = layer(output)
            self.attention_probs[i] = weights

        # 2. END YOUR CODE HERE #

        # Pooling by selection the [CLS] token
        pooled_output = output[:, 0] # (bs, dim)

        # Classification
        # 3. START YOUR CODE HERE #

        h = self.pre_classifier(pooled_output)
        h = nn.functional.relu(h)
        logits = self.classifier(h)

        # 3. END YOUR CODE HERE #

        return logits

```

```

model = DistillBertAttention(config)
state_dict = torch.load('distilbert.pt')
_ = model.load_state_dict(state_dict)
_ = model.eval()

```

```

[68]: # Predict your output
logits = model(inputs['input_ids'])

```

6 4. Visualize the attention weights (10 points)

Let's now look at what tokens the model selects in its self-attention blocks.

1. Use the tokenizer to map the 'input_ids' back to 'tokens'.
2. Extract attention probabilities for every layer by averaging over the attention heads in each layer. You should get a matrix of size [n_layers x seq_length x seq_length]
3. For each layer plot the resulting attention matrix.

```

[83]: import numpy as np
import matplotlib.pyplot as plt

# YOUR CODE HERE #
mean_weights = [torch.mean(weights, dim=1) for weights in model.attention_probs.
    ↪values()]
print(inputs)
tokens = tokenizer.convert_ids_to_tokens(inputs['input_ids'][0].tolist())
print(tokens)

```

```

{'input_ids': tensor([[ 101, 2023, 2001, 2028, 1997, 1996, 2190, 5691, 1045,
2031, 2412, 2464,
1012, 102]]), 'token_type_ids': tensor([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0]]), 'attention_mask': tensor([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1]])}
['[CLS]', 'this', 'was', 'one', 'of', 'the', 'best', 'movies', 'i', 'have',
'ever', 'seen', '.', '[SEP]']

```

```

[86]: import matplotlib.pyplot as plt
import torch
import numpy as np

def plot_attention_heatmap(weights: torch.Tensor, tokens: list[str]):
    """
    Renders a 2D heatmap for a specific attention head.

    Args:
        weights: torch.Tensor of shape (seq_len, seq_len)
        tokens: List of strings representing the sequence tokens
    """
    # Validation of dimensions

```

```

seq_len = len(tokens)
if weights.shape != (seq_len, seq_len):
    raise ValueError(f"Weight shape {weights.shape} mismatch with token_
↳length {seq_len}")

# Conversion to numpy for matplotlib compatibility
data = weights.detach().cpu().numpy()

plt.figure(figsize=(10, 8))
plt.imshow(data, cmap='viridis', aspect='equal')

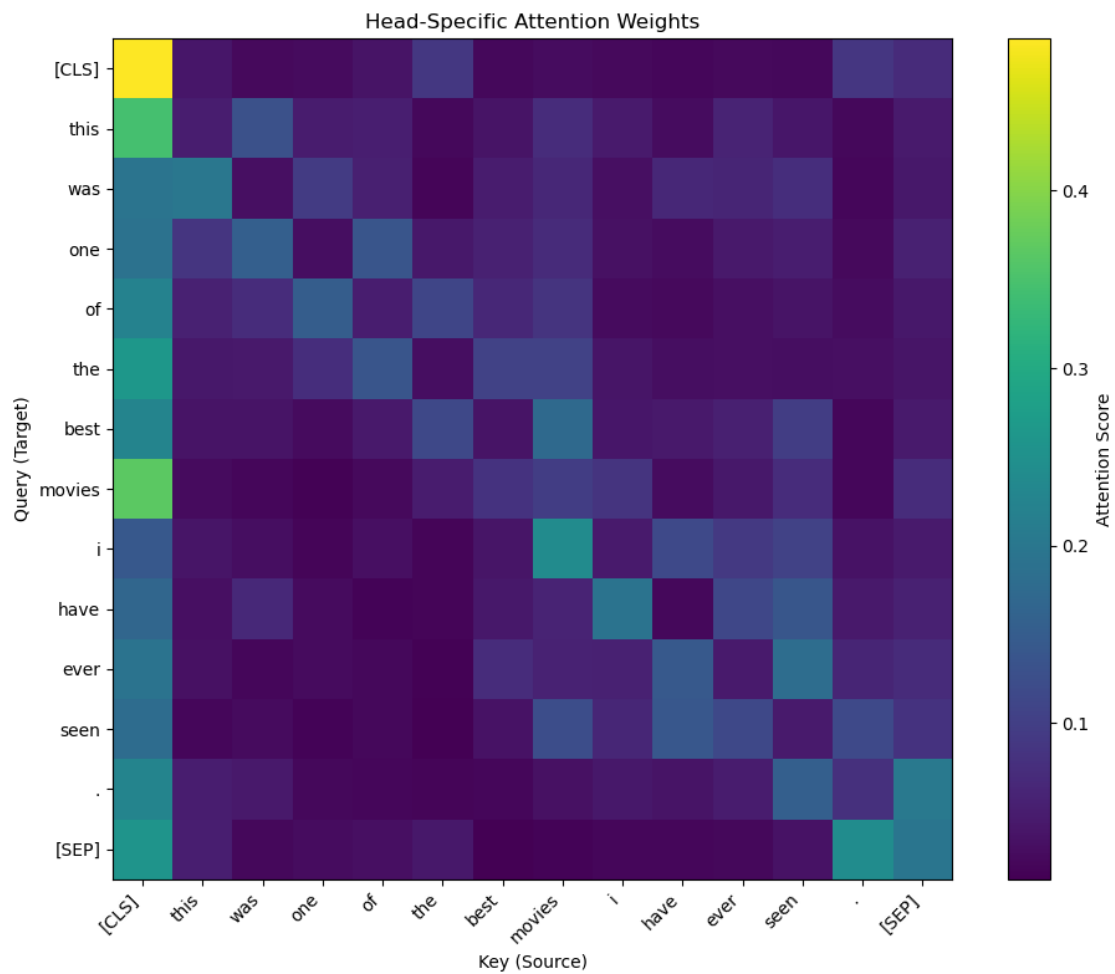
# Axis configuration
indices = np.arange(seq_len)
plt.xticks(indices, tokens, rotation=45, ha='right')
plt.yticks(indices, tokens)

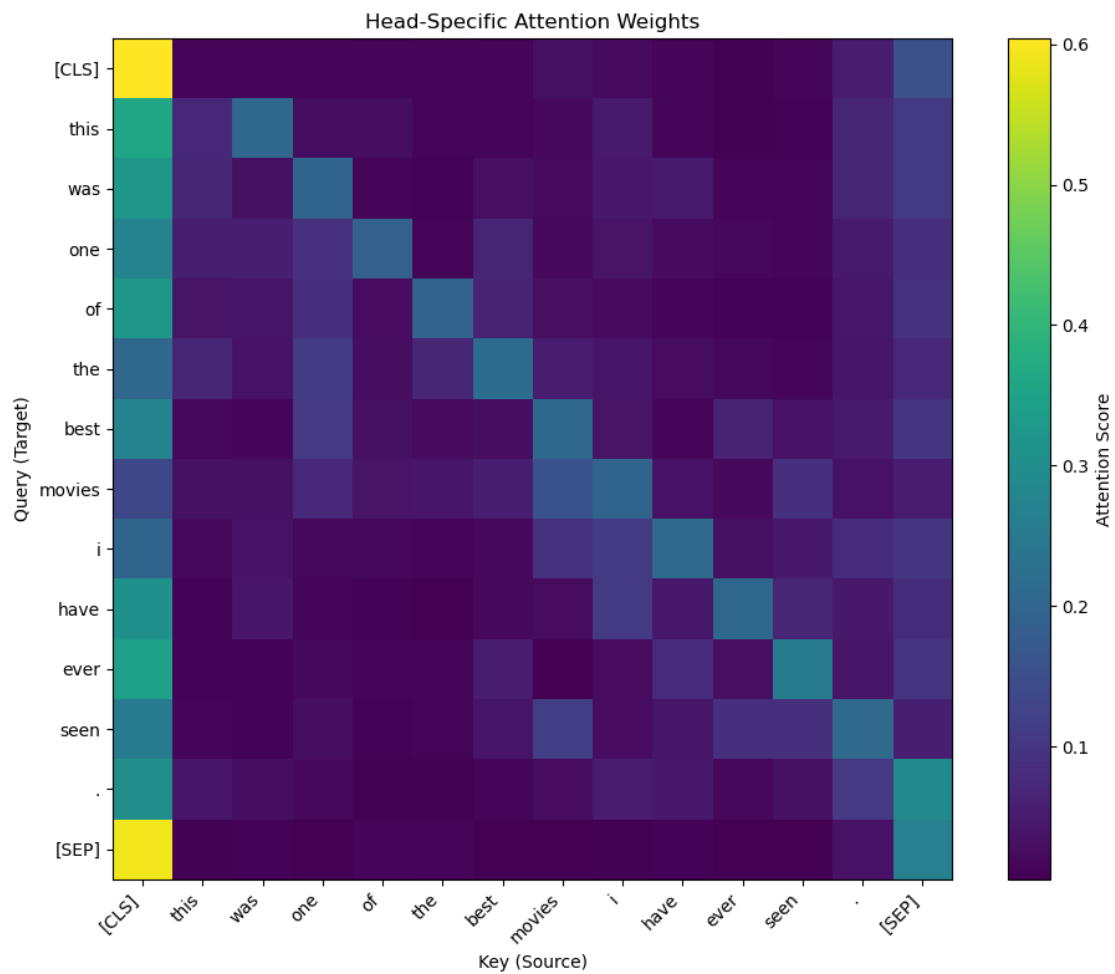
plt.xlabel("Key (Source)")
plt.ylabel("Query (Target)")
plt.colorbar(label="Attention Score")
plt.title("Head-Specific Attention Weights")

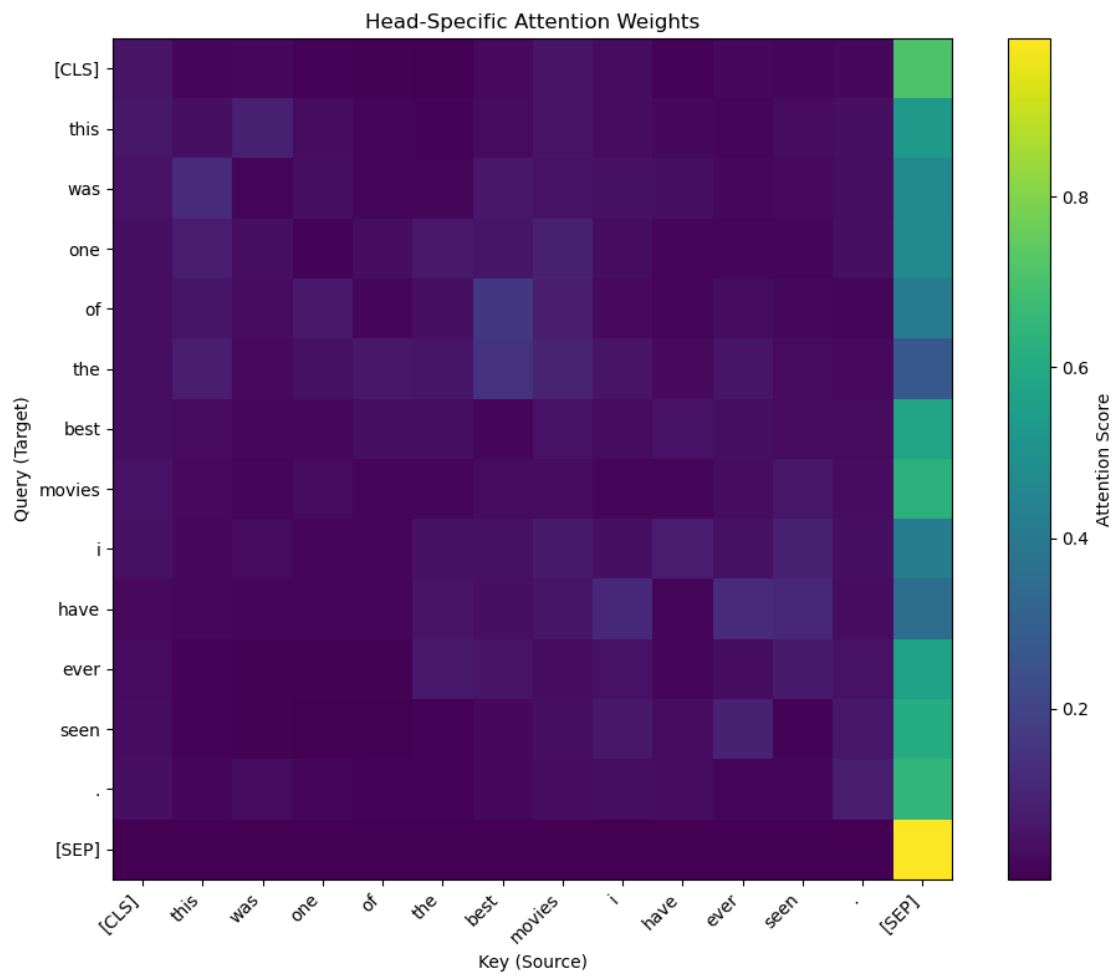
plt.tight_layout()
plt.show()

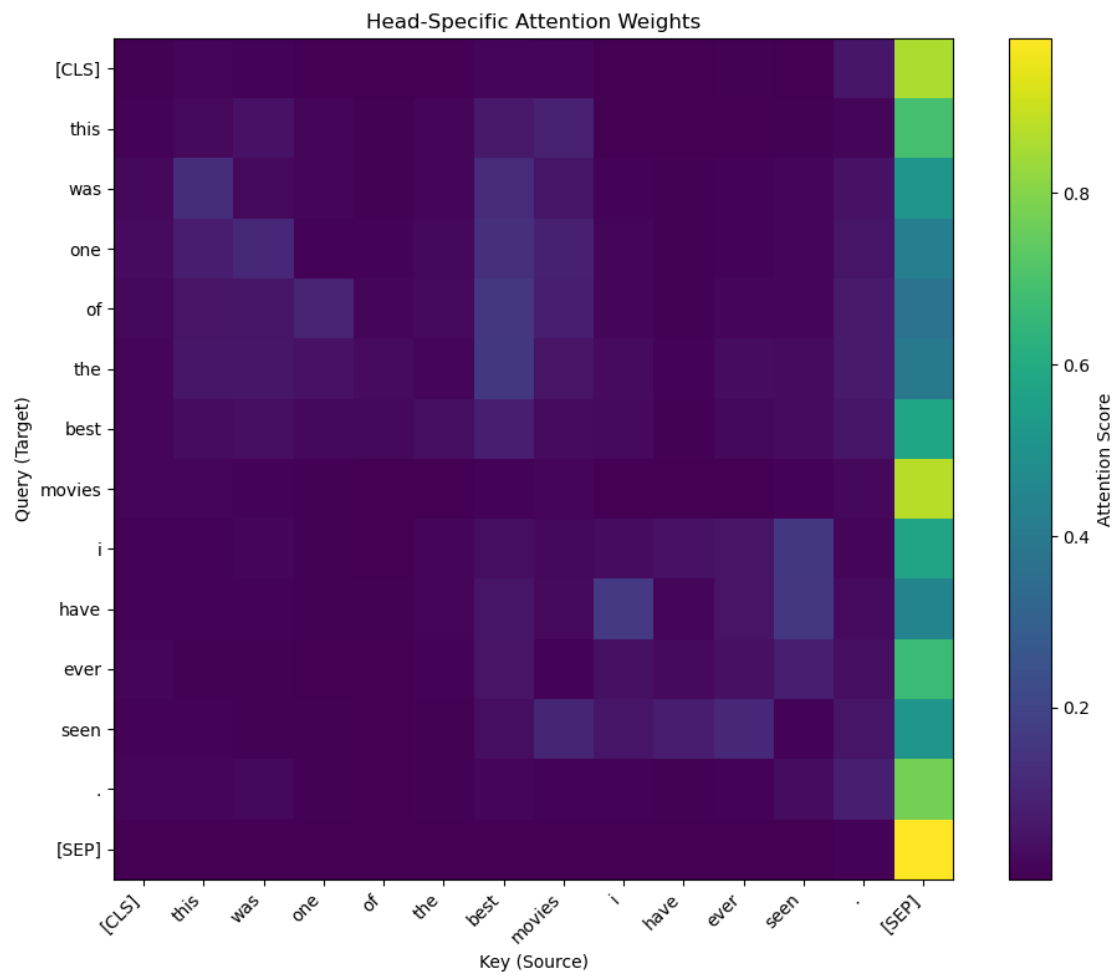
for weight in mean_weights:
    plot_attention_heatmap(weight.squeeze(0), tokens)

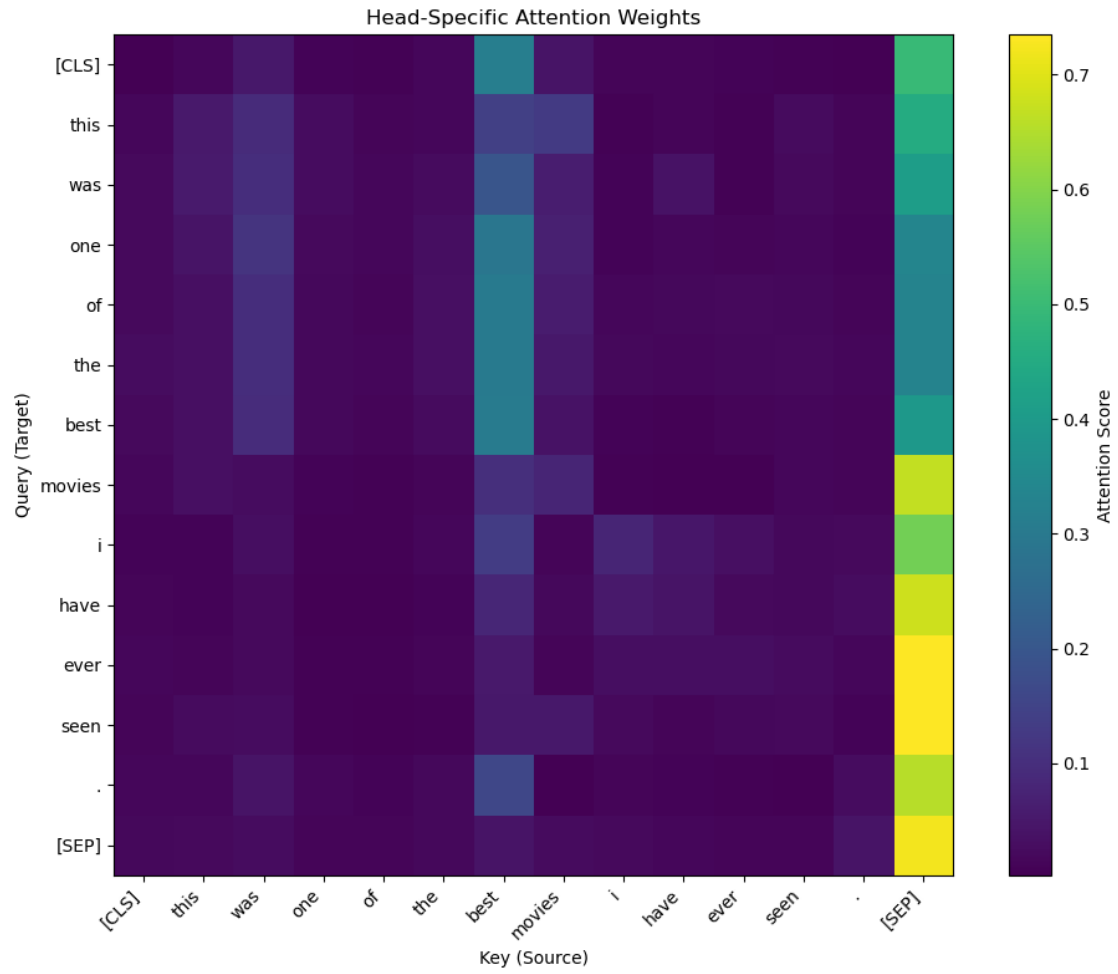
```

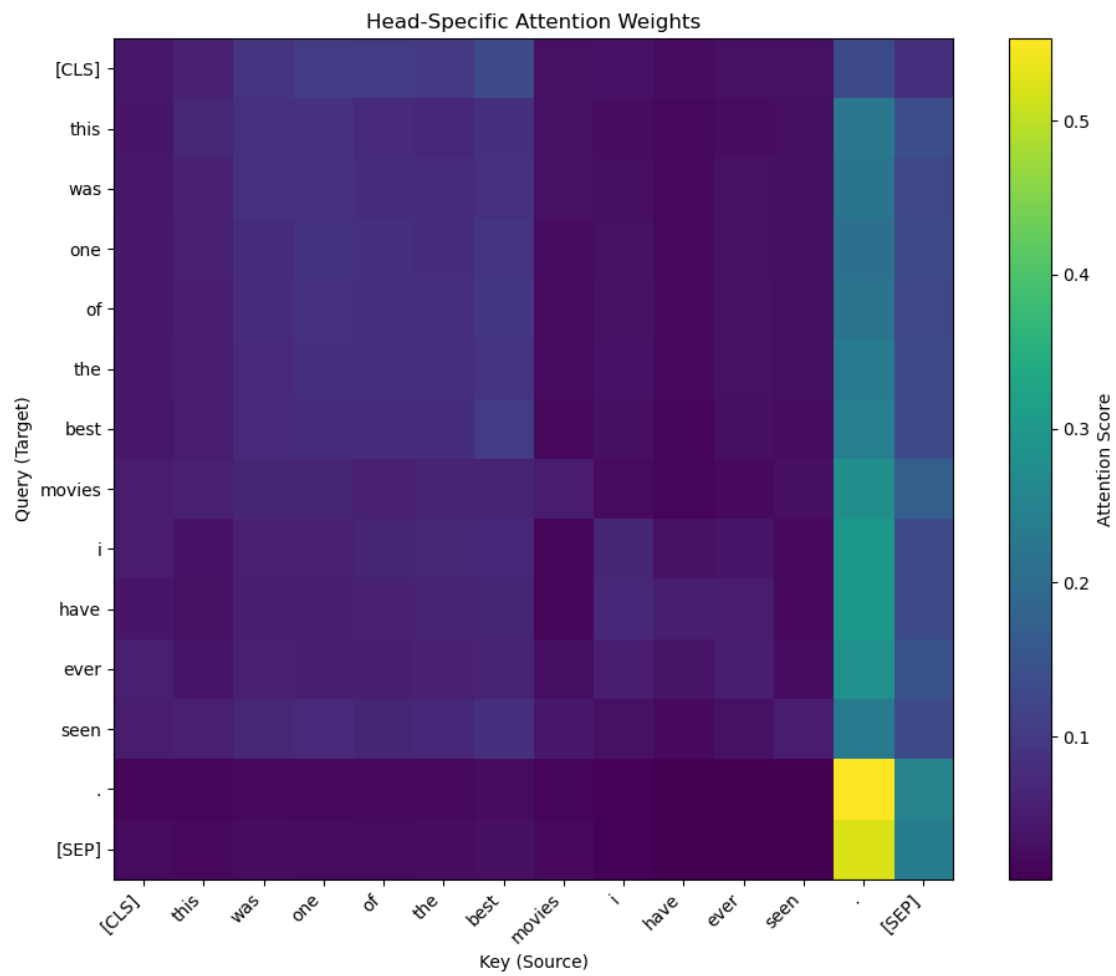












[]: